AI report, sections 1 to 9 (draft)

DD2 Project 1, Option 2: Finding Cells Worth Adding. Group GB2. Repository: github.com/Gad101/cellgap.

1. What the AI did and what we did, part by part

Part	What the AI did	What we did
Understanding the handouts and the pipeline	Explained them step by step when asked	Read the handouts and asked questions until we could explain every step ourselves
Checkpoint 1: AIGs, cuts, truth tables, classes, HD comparison	Wrote the first versions of the scripts	Ran them in the course image, compared with the handout's Tables 1, 2 and 4, and fixed the -zinit and extra opt_clean difference (M-05)
Checkpoint 2: cost model and candidate ranking	Wrote the first versions and the tests	Ran make test, remapped the top cells with Yosys, read and explained the code before using it
Checkpoint 2: Liberty writer, mapping, equivalence check, estimated Table 5	Wrote the scripts	Ran them in the course image, checked 32 of 32 mappings with ABC, fixed the equivalence reference (M-09)
Checkpoint 3: greedy final set, curve, final Table 5, 30% test, instruction-set comparison, full-flow demo	Wrote the scripts and the tests with their broken cases	Chose the split of work, ran the flow in a Docker image with Yosys 0.33, confirmed our teammate's numbers reproduced, fixed a rule violation (M-15), committed
Environment and git	Suggested commands	Built the image, fixed the links to the pack, PDK and tutorials, freed disk space, committed and pushed

Decisions that stayed with us were the split of work, what counts as “reproduced” (the handout's Tables 1, 2 and 4), whether a result was good enough to report, and what to put under Limits.

Decisions I made myself, in the chat with the assistant:

- I decided to have the AI write the checkpoint 3 code (greedy final set, curve, final Table 5, 30% test, instruction-set comparison), and I chose to add cellweaver as the source of real cells once it was released.
- I decided to run the flow natively in Docker rather than rely on the AI's sandbox, because the course requires native Yosys 0.33. I built the extra image with Yosys, fixed the missing links by hand, and freed disk space to make room.
- Some of the software on my Mac was not compatible with the course flow, so I updated it. Docker was installed but its daemon was not running and disk space was short. The course image had no Yosys and no numpy, which made the first run fail, so I built a derived image (dd2-flow) with Yosys 0.33, numpy and iverilog. Earlier, the WebAssembly Yosys build

(yowasp) had given different areas from the handout (M-04), which is why native Yosys 0.33 is required. After the update, the numbers matched the handout and my teammate's.

- I reverted the machine-specific noise in results/ only after confirming with git diff --ignore-cr-at-eol that nothing had changed.
- I decided what to commit and push, and when to push it.
- I decided that the “tested and working” claim needed a real test run first, so I asked whether the code was fully tested before treating the code part as done.
- I decided to prepare for the oral exam by having the whole project explained from the bottom up and by doing a mock exam, so I can explain every part myself.

In checkpoint 1 I helped check the work: the AIG, cut and class results were compared against the handout’s tables before we moved on.

In checkpoint 2 I did Part A: the cost model, the area model and the candidate ranking (costmodel.py and candidates.py).

- The AI wrote the first versions of both files and their tests. I ran make test and ran the flow myself.
- I checked the top-ranked cells by remapping them with Yosys instead of trusting the estimate.
- I read and explained the code before using it. The check costmodel.verify, which simulates the formula for all 3,984 four-input classes, is what gave me confidence in the cost model.

2. Tools, models and cost

We used Claude in chat and in a coding environment. The log records claude-opus-5-5 for most of checkpoint 2's rows and claude-sonnet-5-5 for checkpoint 3. I used Claude Sonnet 5.5 for my work, including the Part A rows that the log records only as “Claude (chat)”.

I use a paid Claude subscription, so there was no per-call cost.

No AI model runs when our tool runs, and no API key is in the repository.

3. How we described tasks, and how we checked the output

We gave the assistant the handout and AGENTS.md first. AGENTS.md says how the project is checked and what the assistant must not do. We described tasks one step at a time, using the handout's step numbers, for example “step 10: select the final set greedily”. Every request was logged in ai/use_log.md with what we kept or changed and how we checked it.

Pass or fail came from scripts, never from the assistant's opinion:

- **Handout targets.** Tables 1, 2 and 4 must be reproduced, never adjusted.
- **The cost model.** costmodel.verify simulates the formula of all 3,984 four-input classes.
- **The Liberty file.** Yosys must read 189 + N cell types, and each new cell's function is compared with its request.
- **Every mapped netlist.** ABC proves each one equals the original design (cec, or dsec when there are flip-flops). All 32 estimated-library mappings and all the final-set mappings pass.

- **Estimates are never trusted.** The final set is chosen by remapping with Yosys, so a cell stays only if the measured area drops.
- **Reproduction on a second machine.** Our teammate's mapping areas matched on the other member's machine.
- **Tests.** make test runs 63 tests in the course image, all passing.

4. How we applied the rules of section 4.3

1. **Pass or fail from scripts.** make test, the equivalence checks, the Liberty import count and the comparison with the handout's target numbers.
2. **Every number comes from a file the flow wrote.** All results are under results/. The totals differ by 0.1 um2 between the greedy log and Table 5, because the log sums unrounded areas and Table 5 sums areas rounded to 0.1.
3. **Every check fails on a deliberately broken case.** There are broken cases in tests/broken/ and inline in tests/test_candidates.py, test_liberty_with_new_cells.py, test_mapping_equivalence.py, test_gate_level_testbenches.py and test_final_set.py. Examples: a cell that makes area larger is dropped, a gain below rounding is dropped, a failed equivalence is marked FAIL, and a missing or unbalanced Liberty cell group is refused.
4. **The AI must not change check scripts, broken cases or the PDK.** AGENTS.md rule 1 says so. We had one violation: in checkpoint 2 the assistant added tests/broken/wrong_cost_expression.json. It noticed this on re-reading AGENTS.md before checkpoint 3 and told us. We removed the file and wrote that broken case inside tests/test_candidates.py (mistake M-15). After that, the assistant did not edit check scripts itself, but it proposed edits to two protected files (the test line in the Makefile and the broken case in tests/test_candidates.py) and I reviewed and applied them myself. We list this openly as a grey area of rule 1, because the content came from the AI.

   We left redundancy.py and make check unchanged. redundancy.py has no commits after its first version, and the Makefile check target is the same as at checkpoint 2 (checked with git log and git diff cp2). Two problems that the assistant flagged stay open and are listed under Limits: redundancy.py prints SELFTEST PASS next to a MISMATCH row (Table 4: new_one_stage_small is 6 against the handout's 5), and make check can accept a WebAssembly Yosys build.
5. **Every member can explain every part of the code.**

   To prepare, I made a sheet that gives a short description of what each code function does and the file it is in. I use it to find and explain any part of the code quickly, and I also did a mock oral exam and had the pipeline explained from the bottom up.

5. The five most instructive AI mistakes

The full rows are in ai/mistakes_log.md. For each one: what went wrong, how we caught it, and whether a weaker check would have missed it.

1. **M-05: the AIG no longer matched the handout.** An early "fix" removed -zinit and added an extra opt_clean, so the AIGs had exactly 3 fewer AND nodes per flip-flop than Table 2 (cpu8:

1,019 against 1,265). We caught it by comparing design_stats.csv with Table 2; the gap was 3 × flip-flops on every design. A weaker check would have missed it: the cut check passes on any AIG, and only the handout number exposed it.

2. **M-04: the wrong Yosys build.** Table 1 was run with the WebAssembly yowasp-yosys, on the assumption that the same version string gives the same results. Script A differed by up to 6.6% (fir10). The TA reran our scripts with native Yosys 0.33 and got the handout numbers, and we confirmed it in the course Ubuntu 24.04 image. A weaker check would have missed it: the version check passes for both builds, and script B matched on four designs, which made the script A gap look like noise.
3. **M-06: ABC's choice nodes left in the AIG.** The after-ABC AIG written straight after dch kept dangling choice nodes (cpu8: 1,959 ANDs instead of 1,109). We caught it because ABC's print_stats showed “choice = 262”. A weaker check would have missed it: the cut check passes on dangling nodes too.
4. **M-09: the equivalence check was wrong, not the netlist.** The first version compared with the raw RTL and paired flip-flops by order, and it reported even the baseline cpu8 netlist (no new cells) as not equivalent. We caught it by running the check on a baseline netlist first, which must pass. A weaker check would have missed it: run only on new-cell netlists, the failures would have been blamed on the new cells.
5. **M-14: the saving estimate was 4× too optimistic.** The first estimate counted every node of an unoptimised AIG cone (top candidate 985 against 250 um2). We caught it by noticing that the top cell needed only 3 ANDs while the cones averaged more. All tests passed, so tests alone would have missed it. The later result makes the lesson stronger: in checkpoint 3 the cell ranked first by estimated saving (feefa, 36.74 um2) made the real mapped area larger (+37.5 um2) and the greedy rule dropped it. Only 3 of 11 candidates survived (results/greedy_log.csv).

Two mentions that did not make the five: M-08 (Yosys reads a Liberty file with a missing brace without an error and imports only part of it; the 189 + N count catches it) and M-15 (the rule violation described in section 4).

6. A decision we would never hand to an AI

Whether a new cell is worth adding to the library. An AI can only estimate. The real answer is what ABC does when the cell is in the Liberty file.

The check that catches an AI getting it wrong: the greedy rule keeps a cell only if the best-of-both total area of the six required designs drops in a real Yosys remap, and every mapped netlist is then proven equivalent to the RTL with ABC. The feefa result (estimated saving, real loss) is the evidence that the AI's estimate cannot make this decision.

7. Where AI saved time and where it cost time

It saved time on: writing the pipeline scripts and the tests with their broken cases; explaining the handouts and the pipeline at several levels of detail; and the checkpoint 3 scripts (greedy, curve, final Table 5, 30% test, instruction-set comparison), which were written in one session and run in about 10 minutes of flow time.

It cost time on: the WebAssembly versus native Yosys difference (M-04) and the -zinit issue (M-05), each of which cost debugging time because the AI's first assumption was wrong; the equivalence check bug (M-09); claims that had to be corrected (a wrong date, a zip file that was never delivered, a wrong guess about why AND counts differed); one rule violation that had to be cleaned up (M-15); and the environment setup in Docker, where the course image had no Yosys and I had to build a Yosys 0.33 layer and fix links by hand.

These hours are my own rough estimates, not measured times. The “without AI” figures are guesses about how long the work would have taken me.

Estimated time saved: about 40 to 60 hours.

Task	Without AI	With AI	Saved
Checkpoint 1 scripts (AIG reader, cuts, truth tables, classes, HD comparison)	12 to 16 h	3 to 4 h	9 to 12 h
Checkpoint 2 scripts (cost model, ranking, Liberty writer, mapping, equivalence check, Table 5)	15 to 20 h	4 to 6 h	11 to 14 h
Checkpoint 3 scripts (greedy, curve, final Table 5, 30% test, instruction-set comparison)	10 to 14 h	2 to 3 h, plus 10 min of flow time	8 to 11 h
Tests with broken cases (about 63 tests)	6 to 8 h	1 to 2 h	5 to 6 h
Explaining the handouts and the pipeline	4 to 6 h	1 h	3 to 5 h

Estimated time lost: about 8 to 14 hours.

Task	Time lost
WebAssembly versus native Yosys (M-04)	2 to 4 h
-zinit and extra opt_clean (M-05)	2 to 3 h
Equivalence check bug (M-09)	1 to 2 h
Docker setup, links and disk space	2 to 3 h
Corrections and the M-15 clean-up	1 to 2 h

The net effect was clearly positive, but the lost time was real. It was concentrated in assumptions the AI made without a check (tool versions and AIG flags), which is the lesson in section 9. These figures cover my parts only; my teammate should add his own.

8. Agentic system (section 4.2)

Not applicable. We built the tool ourselves with AI help (section 4.1). No agent runs when the tool runs, so there is no agent diagram, no per-run token limit, no agent-versus-fixed-pipeline comparison and no model settings to report.

9. What we would and would not hand to an AI next time, and why

We would hand over, with checks: scaffolding scripts, tests and their broken cases, explanations, repetitive file formats (the Liberty writer, tables) and first drafts of logs.

We would not hand over: choosing the target numbers or what counts as reproduced; editing check scripts, broken cases or the PDK; deciding from an estimate whether something is an improvement; assumptions about tool versions (the Yosys build, the AIG flags), because a plausible-sounding assumption was wrong more than once; and stating what a run proved before the check has run.

The reasons are the five mistakes above. The rule of thumb that came out of them: the AI is useful when a script can say pass or fail, and risky when only its own judgement stands behind a statement.